Table of Contents

Preface **1**
Three reasons 1
Learning RISC-V Assembler is future-proof (and fun) 1
Politics 2
The foundation 3
So why should I read this book? 3
RISC-V and bare metal? 4
What is in the book? 5
Part 1: The basics 5
Part 2: Time to interrupt 6
Part 3: Challenge me with DMA, PIO and two cores 6
What about the hardware costs? 7
Where is the code and how to use this book? 9
What about AI? 10
What about errors? 11
Thanks 11
1. Beginning the journey **12**
1.1. Getting the hardware ready 12
1.1.1. The Debug Probe 12
1.1.2. The Pico2 (the workhorse) 16
1.2. Installing the RISC-V toolchain 16
1.3. Installing OpenOCD 18
1.4. Let’s toggle a GPIO output (RISC-V) 19
1.4.1. The Assembly code 19

1.4.2. The Linker script . . . 21
1.4.3. Creating the binary file and programming it using OpenOCD . . . 22
1.5. Analyzing the source code . . . 24
1.5.1. Directives, Sections, Data types and Definitions . . . 24
1.5.2. The weird block . . . 26
1.5.3. Our Entry point "_start" . . . 27
1.5.4. The RISC-V ABI Register conventions . . . 28
1.5.5. Flowchart overview . . . 30
1.5.6. Setting up a GPIO pin in the "setup" part . . . 31
1.5.7. Prepare toggle mask for GPIOx . . . 38
1.5.8. Around and around "mainloop" . . . 39
1.5.9. And if we have no oscilloscope . . . 41
1.6. Assemble, link, and program . . . 43
1.6.1. Assembling . . . 44
1.6.2. Linking . . . 47
1.6.3. Programming . . . 50
1.7. Final thoughts on chapter 1 . . . 51
2. Kill the bugs, align it and tidy up . . . 53
2.1. Kill the bugs with GDB . . . 53
2.1.1. What is GDB? . . . 53
2.1.2. Using GDB . . . 54
2.2. The elephant in the room (alignment) . . . 57
2.2.1. The misaligned LED . . . 59
2.2.2. Aligning things with Assembler directives . . . 60
2.2.3. Stack alignment . . . 62

2.3. Tidying up by using the preprocessor . . . 62
2.4. Splitting the code into different files . . . 63
2.4.1. The weird block in "rp2350_riscv_image_def_block.S" . . . 64
2.4.2. Register definitions in "rp2350_definitions.S" . . . 65
2.4.3. Macros in "rp2350_macros.S" . . . 66
2.4.4. A delay subroutines library "srs_delays.S" . . . 69
2.5. Subroutine library to initialize GPIO pins "srs_gpios.S" . . . 70
2.5.1. Calling a subroutine in a subroutine . . . 71
2.5.2. Calculating in Assembler . . . 72
2.5.3. Bringing it together . . . 77
2.6. Subsystem Resets lib. "srs_resets.S" . . . 80
2.7. The main program "02_toggle_tidy.S" . . . 84
2.8. The "ass_link_program_min.sh" script . . . 86
2.9. What to do if everything goes wrong . . . 88
2.10. Final thoughts on chapter 2 . . . 88
3. The clock is ticking . . . 90
3.1. Overview . . . 90
3.1.1. Preparing the ground . . . 92
3.2. Watching the clock . . . 93
3.3. Switching to the crystal oscillator . . . 99
3.4. Multiplying the frequency with a PLL . . . 102
3.4.1. Creating the system clock frequency of 150 MHz . . . 103
3.4.2. Creating the USB frequency of 48 MHz . . . 107
3.5. Peripheral clock generator . . . 110
3.6. ADC clock generator . . . 112
3.7. Init clocks . . . 113

3.8. LED in hyperstress (hyperspace?) ... 113
3.9. "00_template.S" ... 115
3.10. Final thoughts on chapter 3 ... 116
4. Serial ... **117**
4.1. Overview ... 117
4.2. Initializing the UART ... 120
4.2.1. Get an UART clock ... 125
4.2.2. Deassert UART resets ... 125
4.2.3. Set two GPIOx to UART TX and RX ... 125
4.2.4. Clear the RX FIFO buffer ... 127
4.3. Read and writing bytes from/to serial ... 129
4.3.1. Read a byte from UART0: "uart0_read_byte" ... 129
4.3.2. Write a byte to UART0: "uart0_write_byte" ... 131
4.4. Echo, Echo, Echo ... 133
4.4.1. Main program "04_serial_echo.S" ... 133
4.4.2. Serial hardware connections ... 133
4.4.3. The terminal emulator software ... 137
4.5. Serial.print() ... 139
4.5.1. Subroutine "uart0_print_string" ... 140
4.5.2. Serial.println() ... 142
4.6. Peeking at register and memory ... 143
4.6.1. Let’s start with a single byte: "uart0_print_byte_hex" ... 143
4.6.2. Print two bytes (16 bit): "uart0_print_2byte_hex" ... 146
4.6.3. Print a word: "uart0_print_4byte_hex" ... 147
4.6.4. In decimal: Subroutine "uart0_print_byte" ... 149
4.6.5. In decimal: Subroutine "uart0_print_2byte" ... 150

4.6.6. In decimal: Subroutine "uart0_print_4byte" 151
4.7. What about UART1? 152
4.8. Final thoughts on chapter 4 152
5. "Make It": Pico2Terminal 155
5.1. Using an HDMI monitor 156
5.1.1. Hardware for the HDMI Pico2 157
5.1.2. Arduino Software for the HDMI Pico2 159
5.1.3. Assembler software for the main Pico2 162
5.2. Memory output on the HDMI monitor 163
5.2.1. Display memory byte-wise 163
5.2.2. Display memory word-wise 166
5.2.3. Display memory half-word-wise 168
5.3. Using a Keyboard 170
5.3.1. Hardware for the USB keyboard Pico(2) 170
5.3.2. Arduino software for the USB keyboard Pico(2) 172
5.4. Terminal 176
5.4.1. Hardware for the terminal (3 Picos) 176
5.4.2. Assembler software for the main Pico2 179
5.5. Final thoughts on chapter 5 183
6. Digital and analog input 185
6.1. GPIOx as digital input 185
6.1.1. Pushing the button: Pull up or Pull down? 185
6.1.2. Initializing GPIOx as digital input 188
6.1.3. Read a digital input signal on GPIOx 192
6.2. Analog input using an ADC 194
6.2.1. Hardware: Circuit with potentiometer 196

6.2.2. Initializing the ADC ... 197
6.2.3. Initialize GPIOx as an ADC input ... 199
6.2.4. Select the input channel ... 201
6.2.5. Get a one-shot sample ... 202
6.2.6. Free-running mode ... 205
6.2.7. Reading the system voltage on Pico2 ... 207
6.2.8. Using the internal temperature sensor ... 208
6.3. Final thoughts on chapter 6 ... 211
7. Timers and analog output (PWM) ... 212
7.1. Timer/Counter overview ... 212
7.2. The two system timers TIMER0 and TIMER1 ... 213
7.2.1. Deassert the resets ... 214
7.2.2. Ticks as timer source ... 215
7.2.3. Measure the real time that the code needs ... 218
7.2.4. Subroutines "micros_" and "millis_" ... 219
7.2.5. New delay subroutines ... 222
7.3. RISC-V platform timer ... 223
7.3.1. So how do we start the timer? ... 223
7.3.2. Read the RISC-V platform timer ... 225
7.4. PWM ... 227
7.4.1. PWM Channels ... 228
7.4.2. How do we create a pulse width modulation? ... 229
7.4.3. Calculating the channel base address ... 233
7.4.4. Initializing the PWM ... 235
7.4.5. Dimming an LED ... 238
7.4.6. Using a wavetable to play a sound ... 239

7.4.7. Playing several wavetables in succession 247
7.5. Final thoughts on chapter 7 250
8. Interrupts 252
8.1. Overview … 252
8.1.1. … and a bunch of new terms 253
8.2. RP2350's Hazard3 Interrupts 257
8.2.1. The interrupt CSR (register) 258
8.2.2. Control and status registers (CSRs) instructions 259
8.2.3. External interrupts use the "Xh3irq" interrupt controller 260
8.2.4. What happens when an interrupt occurs? 263
8.2.5. How to activate an interrupt 264
8.2.6. The Interrupt Service Routine ISR 266
8.3. An alarming Rpt interrupt 269
8.3.1. Handling the Rpt time and compare register 269
8.3.2. Subroutines to use the interrupt CSR 270
8.3.3. Initializing the Rpt interrupt 272
8.3.4. Writing the Interrupt Service Routine ISR 272
8.3.5. Putting everything together 273
8.3.6. Adding a push-button to set the alarm 275
8.4. External interrupts: GPIO interrupt 276
8.4.1. GPIO interrupt register 277
8.4.2. Calculating the right GPIO interrupt register and bitmask 278
8.4.3. More subroutines to manipulate interrupt CSR bits 280
8.4.4. Initializing the GPIO interrupt 282

8.4.5. The GPIO Interrupt Service Routine . . . 284
8.4.6. The mainloop of "08_interrupts_gpio.S" . . . 285
8.5. External UART interrupt . . . 286
8.5.1. Initializing the UART interrupt . . . 287
8.5.2. The UART Interrupt Service Routine . . . 288
8.5.3. The mainloop of "08_interrupts_uart.S" . . . 289
8.6. Playing a MIDI song with DDS and an RPT interrupt . . . 289
8.6.1. DDS overview . . . 289
8.6.2. MIDI notes . . . 292
8.6.3. Playing some notes . . . 294
8.6.4. The Interrupt Service Routine for DDS . . . 294
8.6.5. The main program "08_interrupts_dds.S" . . . 296
8.6.6. Playing a MIDI song . . . 297
8.6.7. The subroutine "midi_play_song_dds.S" . . . 299
8.6.8. The main program "08_interrupts_dds_song.S" . . . 301
8.7. Direct vs Vectored Interrupt Handling . . . 301
8.8. Final thoughts on chapter 8 . . . 302
9. "Make It": Pico2Pong . . . 303
9.1. Building the hardware . . . 303
9.1.1. The circuit . . . 303
9.1.2. The housing . . . 304
9.1.3. The paddles . . . 305
9.2. Arduino Software for the HDMI Pico2 . . . 306
9.3. The game . . . 311
9.3.1. Game subroutines . . . 313
9.3.2. Play a MIDI note non blocking . . . 325

9.3.3. Main program "pico2pong.S" 326
9.4. Final thoughts on chapter 9 329
10. Memory and DMA 331
10.1. The memory map 331
10.2. Stack 335
10.3. The linker script and boundaries 339
10.4. Copying ".data" and ".bss" section to SRAM 343
10.4.1. A new template 00_template_2.S 346
10.5. DMA: Copy a memory block m2m 348
10.5.1. Initialize DMA channel x 351
10.5.2. Enable DMA channel x and set the data size 353
10.5.3. Subroutine to copy data memory to memory 354
10.5.4. Helper functions to create tables 355
10.5.5. Main "10_dma_m2m.S" 356
10.6. DMA: Memory to PWM 359
10.6.1. Helper subroutine to expand the wavetable to words 359
10.6.2. Subroutine "dma_chx_send_m2pwm_t" 360
10.6.3. Main "10_dma_m2pwm_ring.S" 360
10.7. DMA: Working with two DMA channels 363
10.7.1. Re-triggering a DMA channel via a second DMA control channel 364
10.7.2. Subroutines "dmax2_chx_send_m2pwm" and "dmax2_chx_ctrl_t" 366
10.7.3. Main "10_dmax2_m2pwm.S" 367
10.8. Final thoughts on chapter 10 369
11. PIO State machines 371

11.1. Overview ... 371
11.1.1. The PIO assembler "pioasm" ... 374
11.2. Configuring the PIO block and the state machine ... 376
11.2.1. The 6 state machine registers ... 377
11.3. The PIO library "srs_pio.S" ... 383
11.3.1. Helper routine to calculate PIOx and SMn from SMx ... 384
11.3.2. Helper routine to calculate the offsets for PIOx and SMn ... 385
11.3.3. Initializing the PIO block and the state machine ... 385
11.3.4. Loading a PIO program to the instruction memory registers ... 388
11.3.5. Subroutines to enable and disable a state machine ... 389
11.3.6. Subroutines to change the state machine’s speed ... 390
11.4. New subroutines to handle GPIOs ... 393
11.5. PIO toggle ... 396
11.5.1. Toggle at 2400 Hz ... 396
11.5.2. Main program "11_pio_toggle" ... 399
11.5.3. Blink an LED at 4 Hz ... 400
11.5.4. Blink an LED at 1 Hz ... 403
11.6. WS2812b (Neopixel) interface ... 404
11.6.1. PIO assembler program WS2812b ... 406
11.6.2. Main program "11_pio_ws2812b.S" ... 409
11.6.3. New library "srs_ws2812b.S" ... 410
11.7. 8 Bit parallel output with DMA ... 412
11.7.1. Hardware: Building a DAC ... 412
11.7.2. PIO program to output 8 bit in parallel ... 413

11.7.3. New subroutines for the DMA library . . . 414
11.7.4. The main program "11_pio_dma_outx8" . . . 416
11.7.5. Let’s play our song :) . . . 420
11.8. I2S hardware interface using PIO . . . 423
11.8.1. I2S overview . . . 423
11.8.2. The hardware . . . 425
11.8.3. I2S PIO state machine . . . 428
11.8.4. New wavetables . . . 431
11.8.5. Main program "11_pio_dma_i2s_48k.S" . . . 432
11.9. I2S DDS . . . 435
11.9.1. DMA ping pong . . . 435
11.9.2. Interrupts from DMA . . . 436
11.9.3. The I2S library . . . 440
11.9.4. Main program "11_pio_dma_i2s_dds_48k.S" . . . 444
11.9.5. What about the timing? . . . 447
11.9.6. My square and saw signals have ripples :(. . . 448
11.9.7. High speed with 384 kHz . . . 450
11.10. Final thoughts on chapter 11 . . . 452
12. Two hearts . . . 455
12.1. Overview . . . 455
12.2. Wakey wakey . . . 457
12.2.1. Replacing the arm commands **sev** and **wfe** . . . 459
12.2.2. A subroutine to wake core1 . . . 460
12.2.3. The two cores library "srs_two_cores.S" . . . 461
12.3. Main program "12_two_cores" . . . 463
12.3.1. A new subroutine to blink the LED . . . 464

12.4. Race conditions and spinlocks . . . 465
12.5. Ring my bell (doorbell interrupts) . . . 466
12.5.1. Configuring doorbell interrupts . . . 467
12.5.2. Interrupt service routines . . . 468
12.5.3. Main program "12_two_cores_irq.S" . . . 469
12.6. Using double buffering to pass data . . . 471
12.6.1. The fence instruction . . . 473
12.6.2. Main program: "12_two_cores_irq_pp.S" . . . 475
12.6.3. Subroutine to reduce potentiometer sensitivity . . . 477
12.7. Global Exclusive Monitor . . . 478
12.7.1. Main program "12_two_cores_irq_gem.S" . . . 480
12.7.2. Passing different variables using a pointer . . . 482
12.8. Final thoughts on chapter 12 . . . 483
13. "Make It": Pico2MCWO (Pico2Synth) . . . 485
13.1. Building the hardware . . . 486
13.1.1. The circuit and the BOM . . . 486
13.2. The Interrupt Service Routines . . . 489
13.2.1. The core0 ISR handling DMA ping-pong and doorbells . . . 490
13.2.2. The core1 ISR handling MIDI . . . 492
13.3. New subroutines and a rainbow . . . 495
13.3.1. Parsing a MIDI messages . . . 495
13.3.2. Trimming the potentiometers . . . 498
13.3.3. The new MCWO library . . . 499
13.3.4. Rainbow LUT . . . 506
13.3.5. Fill the I2S DDS buffer and take into account velocity . . . 507
13.4. The main program "13_pico2mcwo.S" . . . 508

13.4.1. Core 0 setup part . . . 509
13.4.2. Core 1 setup part . . . 512
13.4.3. Core 0 main loop . . . 513
13.4.4. Core 1 main loop . . . 514
13.5. Final thoughts on chapter 13 . . . 521
Appendix A: ASCII code . . . 523
Appendix B: Pico2 Memory Map . . . 525
Appendix C: RISC-V Assembler . . . 529
C.1. Instruction cheat sheet . . . 529
C.1.1. Load, store and move instructions . . . 529
C.1.2. Arithmetic instructions . . . 529
C.1.3. Logical Instructions . . . 530
C.1.4. Shift instructions . . . 530
C.1.5. Jump and subroutine instructions . . . 530
C.1.6. Branch instructions . . . 531
C.1.7. Set instructions . . . 531
C.1.8. NOP instruction . . . 532
C.1.9. Privileged instructions for interrupts . . . 532
C.1.10. CSR base instructions and their pseudo-instr. (Zicsr) . . . 532
C.1.11. Load-Reserved (lr.w) and Store-Conditional (sc.w) . . . 533
C.1.12. Bit-manipulation address-generation instructions Zba . . . 533
C.1.13. Bit manipulation instructions Zbs . . . 533
C.1.14. Bit manipulation instructions Zbb . . . 533
C.1.15. AMO (Atomic Memory Operation) instructions (A) . . . 534
C.1.16. Scalar Cryptography Zbkb . . . 535
C.1.17. Code Size Reduction (Compress instr.) Zcmp . . . 535

C.1.18. Code Size Reduction Zbkb . . . 535
C.2. Instruction terminology . . . 536
C.3. RISC-V extensions implemented by Hazard3 . . . 537
C.4. Remarks . . . 537
C.5. Instructions used per chapter . . . 539
Appendix D: Links . . . **540**
Appendix E: Are we hackers? . . . **543**
E.1. Analyzing code from the SDK . . . 543
E.2. Rearranging the code . . . 547
E.3. Countercheck with **GDB** . . . 548